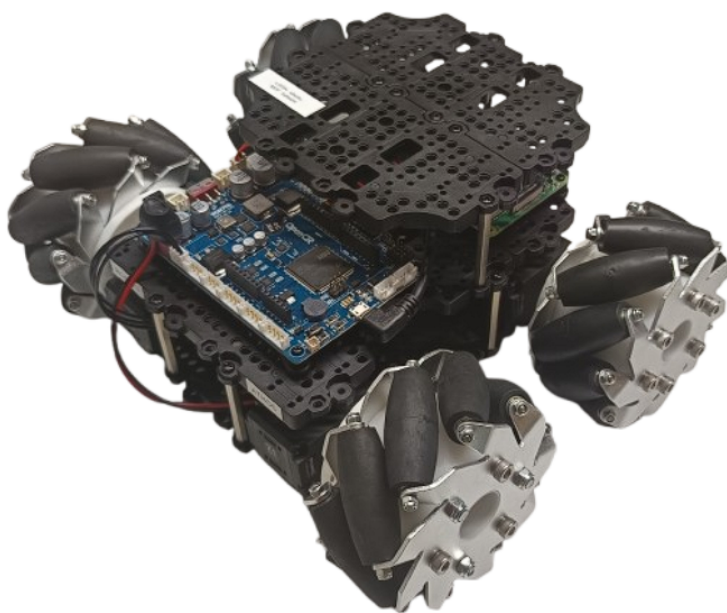


Thibaut Waechter
Promotion : 2024-2025
Année universitaire : 2024-2025

Tutoriel d'installation de ROS2 pour TurtleBot



Organisme d'accueil : iCube
300 Boulevard Sébastien Brandt,
67400 Illkirch



Sylvain Durand
sylvain.durand@insa-strasbourg.fr
03/03/25 - 05/09/25

Table des matières

1	Installation Ubuntu sur Raspberry Pi4	2
1.1	Raspberry Pi Imager	2
1.1.1	Flasher l'OS sur la carte SD	2
1.2	Configuration de la Raspberry Pi	3
1.3	Bibliothèques importantes	3
1.3.1	git	3
1.3.2	net-tools	3
1.3.3	ssh	4
2	Installation de ROS2	5
2.1	Configuration du système	5
2.2	Ajout des dépôts nécessaires et installation des prérequis	5
2.3	Installation de ROS2	6
3	Agent Micro-ROS	7
3.1	Installation	7
3.2	Utilisation	7
3.3	Utilisation via un service	8
4	Programmation de l'OpenCR	10
4.1	Initialisation de l'IDE	10
4.2	Programmation de la carte	10
4.2.1	Inclusion des bibliothèques	11
4.2.2	Création d'un nœud ROS	11
4.2.3	Création d'un publisher et d'un subscriber	11
4.2.4	Callback	11
4.2.5	Configuration d'un timer	12
4.2.6	Callback du timer	12
4.2.7	Configuration de l'exécuteur	12
4.2.8	Boucle principale	12
4.2.9	Programme pour robot Mecanum	12
5	Utilisation de ROS en réseau	13
6	Motion Capture	14
6.1	Installation du package <i>vrpn_mocap</i>	14
6.2	Quality of Service (QoS) sur ROS2	15
7	TF2	17
7.1	Définition	17
7.2	Mise en place de tf2	18
7.2.1	Initialisation de tf2 : Buffer, Listener et Broadcaster	18
7.2.2	Publication de la transformation du robot	18
7.2.3	Transformation d'une commande de vitesse	19

7.2.4	Exemple d'utilisation	20
8	Package <i>Mecanum</i>	22
8.1	Téléchargement du package	22
8.2	Utilisation du package	22
A	Modification des QoS	24
A.1	Python	24
A.2	Matlab/Simulink	25
A.2.1	Matlab	25
A.2.2	Simulink	25
B	Changement de <i>hostname</i> et renommage de l'utilisateur principal	26
B.1	Création d'un utilisateur temporaire	26
B.2	Arrêter tous les processus de l'utilisateur à renommer	26
B.3	Renommer l'utilisateur principal	26
B.4	Modifier la configuration de <i>cloud-init</i>	27
B.5	Modifier les fichiers <i>/etc/hostname</i> et <i>/etc/hosts</i>	27
B.6	Vérification optionnelle de la configuration Netplan	28
B.7	Redémarrage	28

1 Installation Ubuntu sur Raspberry Pi4

1.1 Raspberry Pi Imager

Raspberry Pi Imager est un outil permettant de flasher des images de système d'exploitation sur des cartes S, ce qui permet dans notre cas d'installer Ubuntu sur la carte SD, qui sera ensuite insérée dans la Raspberry PI.

Télécharger le logiciel depuis le site officiel de Raspberry Pi : Raspberry Pi OS – Raspberry Pi : <https://www.raspberrypi.com/software/>.

1.1.1 Flasher l'OS sur la carte SD

- Lancer Raspberry Pi Imager.
- Sélectionner le système d'exploitation :
 - Aller dans *"Other general-purpose OS"* > *"Ubuntu"* > *"Ubuntu Desktop 22.04.03 LTS (64-bit)"*.

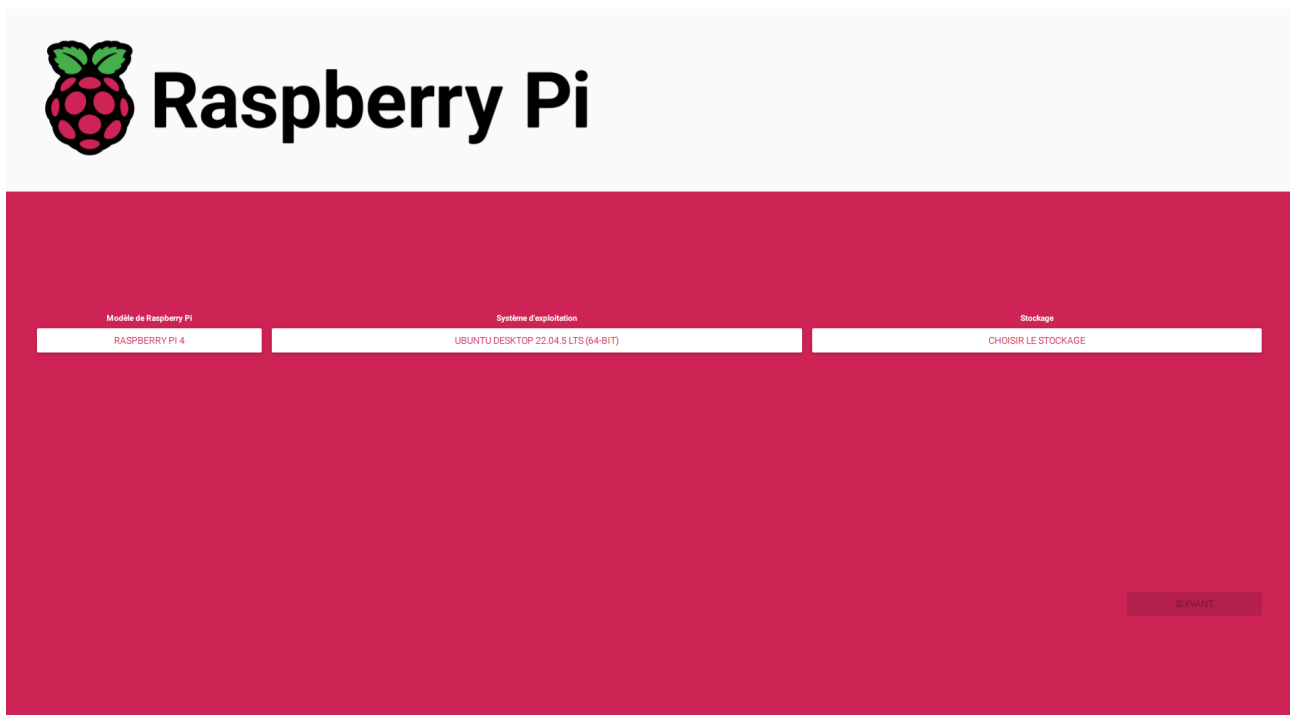


FIGURE 1.1 – Raspberry Pi Imager

- Sélectionner le périphérique de stockage : insérer une carte SD formatée en FAT32.
- Configurer les paramètres, dans ce tutoriel *ubuntu* est choisi comme nom :
 - Nom d'hôte : définir sur *ubuntu.local*.
 - Activer SSH avec authentification par mot de passe.
 - Choisir un nom d'utilisateur et un mot de passe :
 - Nom d'utilisateur : *ubuntu*
 - Mot de passe : *turtlebot*

- Configurer le Wi-Fi avec le SSID et le mot de passe du réseau (s'assurer que le SSID n'est pas caché).
- Définir le fuseau horaire sur *Europe/Paris* et le clavier sur *fr*.
- Écrire l'OS sur la carte SD en cliquant sur le bouton rouge *Write*. Ce processus peut prendre du temps en raison de l'écriture et de la vérification des données.

1.2 Configuration de la Raspberry Pi

Après avoir flashé l'OS, il est alors possible de mettre la carte SD dans la Raspberry Pi, et de l'allumer, ce qui va lancer Ubuntu 22.04 nouvellement téléchargé. Il faut alors suivre les instructions liés au premier lancement, en choisissant le réseau Wifi à utiliser, ou encore le type de clavier.

1.3 Bibliothèques importantes

Tout d'abord, entrer les commandes suivantes :

```
sudo timedatectl set-ntp on
```

Cette commande permet de régler automatiquement l'heure de la Raspberry Pi, via connexion internet. Puis, mettre à jour le système :

```
sudo apt update  
sudo apt upgrade
```

Si l'utilisateur *ubuntu* ne dispose pas des droits administrateurs, les ajouter :

```
sudo usermod -a -G dialout $USER
```

Puis vérifier que l'utilisateur appartient bien au groupe *dialout* avec :

```
groups
```

1.3.1 git

git permet de télécharger des dépôts se trouvant sur Github, ce qui est nécessaire pour télécharger ROS2 par exemple. Pour télécharger *git*, entrer la commande suivante :

```
sudo apt install git
```

1.3.2 net-tools

net-tools est un outil permettant de voir l'adresse IP et l'adresse MAC de la Raspberry Pi. Pour le télécharger, entrer la commande suivante :

```
sudo apt install net-tools
```

1.3.3 ssh

SSH permet de se connecter à distance à la Raspberry Pi. Pour l'installer, exécuter :

```
sudo apt install openssh-server
```

Ensuite, activer le service SSH et s'assurer qu'il démarre automatiquement au démarrage :

```
sudo systemctl enable ssh  
sudo systemctl start ssh
```

Vérifier que le service est bien en cours d'exécution :

```
systemctl status ssh
```

Enfin, pour se connecter en SSH à la Raspberry Pi depuis un autre ordinateur, utiliser :

```
ssh ubuntu@adresse_ip_de_la_pi
```

Remplacer *adresse_ip_de_la_pi* par l'adresse IP de la Raspberry Pi, que l'on peut obtenir avec la commande (après avoir installé *net-tools* :

```
ifconfig
```

ubuntu correspond au nom utilisé sur la Raspberry Pi.

2 Installation de ROS2

Il est possible de retrouver ce tutoriel en cliquant [ici](#). Il est conseillé de suivre le tutoriel sur le site internet plutôt que sur le présent document car il est plus simple de *copier-coller* les commandes (les retours à la ligne, symbolisés par des flèches sur ce document, posent problème).

2.1 Configuration du système

Avant d'installer ROS2, il est nécessaire de mettre à jour le système. Suivre les étapes suivantes :

1. Mettre à jour les paquets et installer les locales :

```
sudo apt update && sudo apt install locales
```

2. Générer les locales pour l'anglais des États-Unis (en_US) :

```
sudo locale-gen en_US en_US.UTF-8  
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
```

3. Exporter la variable d'environnement pour la langue :

```
export LANG=en_US.UTF-8
```

4. Vérifier les paramètres des locales :

```
locale
```

2.2 Ajout des dépôts nécessaires et installation des prérequis

1. Installer les outils nécessaires pour gérer les dépôts logiciels :

```
sudo apt install software-properties-common
```

2. Ajouter le dépôt "universe" :

```
sudo add-apt-repository universe
```

3. Mettre à jour les paquets et installer curl :

```
sudo apt update && sudo apt install curl -y
```

4. Télécharger la clé ROS2 :

```
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/  
→ master/ros.key -o /usr/share/keyrings/ros-archive-keyring.gpg
```

Note : la flèche dans la commande ci-dessous permet de montrer le retour à la ligne involontaire, du à l'écriture de ce tutoriel sur un pdf. Elle est à supprimer dans la commande finale. Cette note s'applique à toutes les commandes du présent document.

5. Ajouter le dépôt ROS2 à la liste des sources :

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/  
→ keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2  
→ /ubuntu $(. /etc/os-release && echo \jammy) main" | sudo tee /  
→ etc/apt/sources.list.d/ros2.list > /dev/null
```

2.3 Installation de ROS2

1. Mettre à jour et améliorer le système :

```
sudo apt update  
sudo apt upgrade
```

2. Installer les paquets de base de ROS2. Deux options sont disponibles selon l'utilisation prévue de ROS2 :

- **Installation Desktop (Recommandée)** : Inclut ROS, RViz, des démonstrations et des tutoriels.

```
sudo apt install ros-humble-desktop
```

- **Installation ROS-Base (Version minimale)** : Inclut les bibliothèques de communication, les paquets de messages et les outils en ligne de commande.

```
sudo apt install ros-humble-ros-base
```

3. Après avoir choisi entre ROS-Base et ROS-Desktop, ajouter l'environnement ROS2 au chemin système :

```
echo source /opt/ros/humble/setup.bash >> ~/.bashrc  
source ~/.bashrc
```

Ensuite, installer colcon :

```
sudo apt install python3-colcon-common-extensions
```


3 Agent Micro-ROS

3.1 Installation

La communication dans ROS2 entre un ordinateur et un microcontrôleur moins puissant est réalisé par un agent Micro-ROS. Cet agent joue le rôle d'intermédiaire, permettant l'intégration de dispositifs à puissance limitée dans un réseau ROS2. Le processus d'installation de ces agents est détaillé sur le site suivant : <https://micro-xrce-dds.docs.eprosima.com/en/latest/installation.html>.

Ouvrez un terminal et exécutez les commandes suivantes :

```
git clone https://github.com/eProsima/Micro-XRCE-DDS-Agent.git
cd Micro-XRCE-DDS-Agent
mkdir build && cd build
cmake ..
make
sudo make install
sudo ldconfig /usr/local/lib/
```

Note : L'opération *make* peut prendre plus d'une heure.

3.2 Utilisation

Une fois l'installation terminée, il est possible de démarrer un agent avec la commande suivante :

```
MicroXRCEAgent serial --dev /dev/ttyACM0 -b 115200
```

Si l'agent est correctement installé, les lignes suivantes doivent s'afficher dans la console :

```
porthos@porthos-desktop:~$ MicroXRCEAgent serial --dev /dev/ttyACM0 -b 115200
[1743593179.525752] info | TermiosAgentLinux.cpp | init | running... | fd: 3
[1743593179.526457] info | Root.cpp | set_verbose_level | logger setup | verbose_level: 4
```

FIGURE 3.1 – Lancement de l'agent

De plus, si l'OpenCR est correctement programmée (voir partie programmation de l'OpenCR) et que la communication est réussie, d'autres lignes peuvent apparaître :

```
porthos@porthos-desktop:~$ MicroXRCEAgent serial --dev /dev/ttyACM0 -b 115200
[1743591052.789937] info | TermiosAgentLinux.cpp | init | running... | fd: 3
[1743591052.790862] info | Root.cpp | set_verbose_level | logger setup | verbose_level: 4
[1743591057.303442] info | TermiosAgentLinux.cpp | fini | server stopped | fd: 3
[1743591057.313895] info | TermiosAgentLinux.cpp | init | Serial port not found. | device: /dev/ttyACM0, error 2, waiting for connec
tion...
[1743591058.320217] info | TermiosAgentLinux.cpp | init | Serial port not found. | device: /dev/ttyACM0, error 2, waiting for connec
tion...
[1743591058.404278] info | TermiosAgentLinux.cpp | init | running... | fd: 3
[1743591060.932232] info | Root.cpp | create_client | create | client_key: 0x5B972078, session_id: 0x81
[1743591060.932437] info | SessionManager.hpp | establish_session | session established | client_key: 0x5B972078, address: 0
[1743591060.950115] info | ProxyClient.cpp | create_participant | participant created | client_key: 0x5B972078, participant_id: 0x000(1)
[1743591060.951625] info | ProxyClient.cpp | create_topic | topic created | client_key: 0x5B972078, topic_id: 0x000(2), partici
ant_id: 0x000(1)
[1743591060.952481] info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x5B972078, subscriber_id: 0x000(4), par
ticipant_id: 0x000(1)
[1743591060.954495] info | ProxyClient.cpp | create_datareader | datareader created | client_key: 0x5B972078, datareader_id: 0x000(6), sub
scriber_id: 0x000(4)
```

FIGURE 3.2 – Lancement de l'agent avec OpenCR

Dans le cas où l'OpenCR est correctement programmé et que l'Agent s'est lancé, mais que les lignes supplémentaires de la figure 3.2 n'apparaissent pas, cela signifie que la communication entre la Raspberry Pi et l'OpenCR ne s'est pas faite. Dans ce cas, il faut réinitialiser l'OpenCR en appuyant sur le bouton *reset* (voir Figure 3.3).

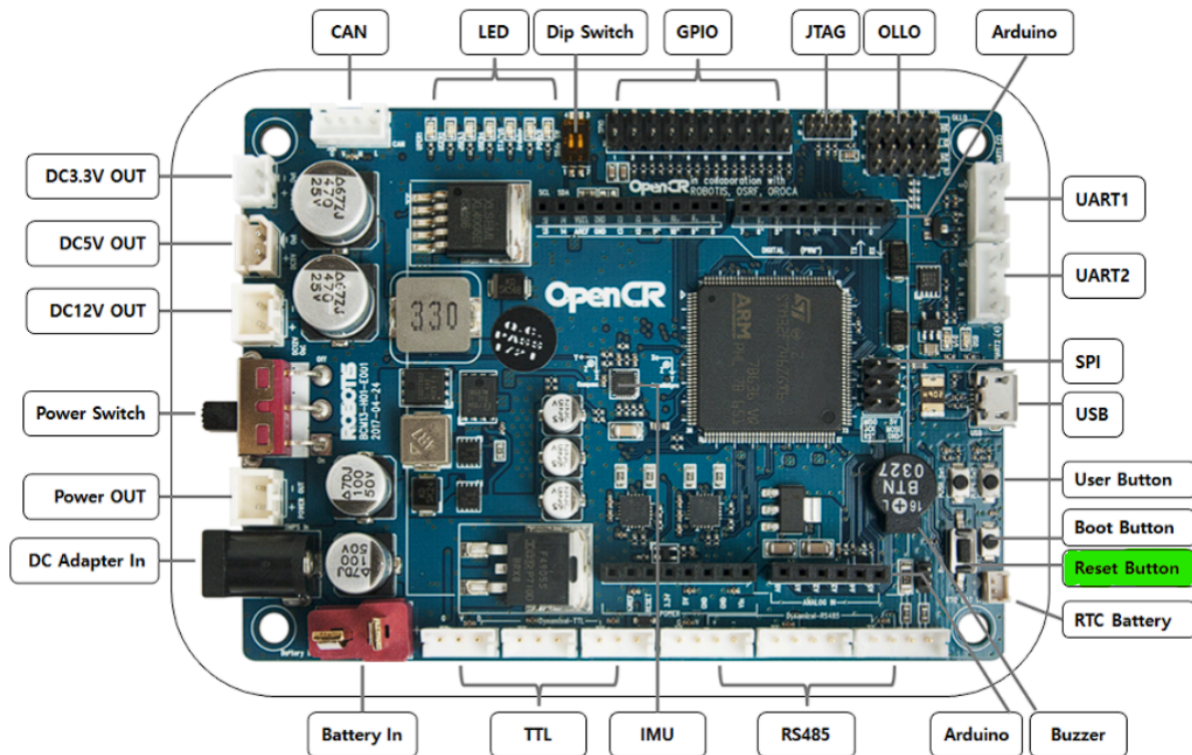


FIGURE 3.3 – Pinout OpenCR

Si tout est correctement configuré, les topics auxquels doit avoir accès l'OpenCR seront visibles en utilisant la commande :

```
ros2 topic list
```

3.3 Utilisation via un service

Un service sur Ubuntu permet d'exécuter automatiquement une commande au démarrage de la Raspberry Pi. Ici, nous souhaitons lancer automatiquement le Micro-Ros Agent.

Créer le fichier de service en exécutant la commande suivante :

```
sudo nano /etc/systemd/system/microxrce.service
```

Y ajouter le contenu suivant :

```
# /etc/systemd/system/microxrce.service

[Unit]
Description=MicroXRCEAgent
After=network.target

[Service]
ExecStart=/usr/local/bin/MicroXRCEAgent serial --dev /dev/ttyACM0 -b
↪ 115200
Restart=always

[Install]
WantedBy=multi-user.target
```

Après avoir créé ou modifié le fichier de service, recharger *systemd* afin de prendre en compte les modifications :

```
sudo systemctl daemon-reload
```

Pour permettre au service de se lancer automatiquement au démarrage du système, exécuter la commande suivante :

```
sudo systemctl enable microxrce.service
```

Pour démarrer immédiatement le service sans attendre le prochain redémarrage du système, exécuter :

```
sudo systemctl start microxrce.service
```

Dans le cas où la Raspberry Pi est redémarrée, ne pas hésiter à regarder si tous les topics utiles au projet (par exemple, ceux auxquelles souscrit ou publie l'OpenCR) sont visibles via la commande :

```
ros2 topic list
```

S'il ne sont pas visibles, appuyer sur le bouton *reset* de l'OpenCR (voir Figure 3.3) pour redémarrer la communication.

D'autres commandes utiles pour gérer le service :

— Stopper le service

```
sudo systemctl stop microxrce
```

— Redémarrer le service

```
sudo systemctl restart microxrce
```

4 Programmation de l'OpenCR

4.1 Initialisation de l'IDE

Pour programmer la carte OpenCR, l'IDE Arduino sera utilisé. Il faut donc le télécharger ici : <https://www.arduino.cc/en/software/>

Une fois l'IDE installé et ouvert, aller dans **File > Preferences**, et ajouter le lien suivant :

https://raw.githubusercontent.com/ROBOTIS-GIT/OpenCR/master/arduino/opencr_release/package_opencr_index.json

Dans la case *URL de gestionnaire de cartes supplémentaires* :



FIGURE 4.1 – URL de gestionnaire de cartes supplémentaires

Plus de détails sont donnés ici : <https://emanual.robotis.com/docs/en/parts/controller/opencr10/>

4.2 Programmation de la carte

La bibliothèque *Micro-ROS-Arduino* (https://github.com/micro-ROS/micro_ros_arduino) est utilisée pour programmer l'OpenCR. Elle permet notamment de communiquer avec la Raspberry PI et le Micro-Ros Agent si cette dernière est branché en USB avec l'OpenCR. Des exemples de code sont disponibles dans **File > Exemples > micro_ros_arduino**.

Ci-dessous est donné une explication de leur fonctionnement.

4.2.1 Inclusion des bibliothèques

```
#include <micro_ros_arduino.h>
#include <rcl/rcl.h>
#include <rcl/error_handling.h>
#include <rcl/rcl.h>
#include <rcl/executor.h>
```

4.2.2 Création d'un nœud ROS

La fonction `rcl_node_init_default` initialise un nœud ROS2, elle est à mettre dans le *setup* :

```
rcl_node_t node;
rcl_support_t support;
RCCHECK(rcl_node_init_default(&node, "micro_ros_arduino_node", "", &
    ↪ support));
```

4.2.3 Création d'un publisher et d'un subscriber

Il faut mettre ces programmes dans le *setup*.

```
RCCHECK(rcl_publisher_init_default(
    &publisher,
    &node,
    ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, Int32),
    "micro_ros_arduino_node_publisher"));
```

Listing 4.1 – Création d'un Publisher

```
RCCHECK(rcl_subscription_init_default(
    &subscriber,
    &node,
    ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, Int32),
    "micro_ros_arduino_subscriber"));
```

Listing 4.2 – Création d'un Subscriber

4.2.4 Callback

Un *callback* est attribué à un *subscriber*, ce qui lui permet de récupérer les données sur un topic.

```
void subscription_callback(const void * msgin) {
    const std_msgs__msg__Int32 * msg = (const std_msgs__msg__Int32 *)msgin
    ↪ ;
    digitalWrite(LED_PIN, (msg->data == 0) ? LOW : HIGH);
}
```

4.2.5 Configuration d'un timer

Permet de programmer un *timer*, qui va organiser l'exécution des tâches à des intervalles réguliers.

```
const unsigned int timer_timeout = 1000; // Intervalle en ms
RCCHECK(rclc_timer_init_default(
    &timer,
    &support,
    RCL_MS_TO_NS(timer_timeout),
    timer_callback));
```

4.2.6 Callback du timer

```
void timer_callback(rcl_timer_t * timer, int64_t last_call_time) {
    if (timer != NULL) {
        RCSOFTCHECK(rcl_publish(&publisher, &msg, NULL)); // Publier le
        ↪ message
        msg.data++;
    }
}
```

4.2.7 Configuration de l'exécuteur

L'exécuteur va, comme son nom l'indique, exécuter les différents *callback*

```
RCCHECK(rclc_executor_init(&executor, &support.context, 1, &allocator));
RCCHECK(rclc_executor_add_timer(&executor, &timer));
```

Par exemple, pour ajouter un *subscriber* à l'exécuteurs :

```
RCCHECK(rclc_executor_add_subscription(
    &executor,
    &subscriber,
    &msg,
    &subscription_callback,
    ON_NEW_DATA));
```

4.2.8 Boucle principale

```
RCSOFTCHECK(rclc_executor_spin_some(&executor, RCL_MS_TO_NS(100)));
```

4.2.9 Programme pour robot Mecanum

Les programmes Arduino pour les robots Mecanum sont disponibles ici : https://github.com/watson67/mecanum/tree/main/holonome_mecanum_ros2. Il faut mettre les 4 fichiers dans un même dossier, ouvrir *holonome_mecanum_ros2.ino* dans l'IDE Arduino puis téléverser.

5 Utilisation de ROS en réseau

Il est fortement recommandé d'utiliser un réseau privé, sur lequel il est possible d'attribuer des adresses IP fixes (routeurs, réseau INSA-Iot). S'assurer que chaque appareil du réseau ait le même *ROS_DOMAIN_ID*. Pour cela, utiliser les commandes suivantes sur chaque appareil :

```
echo export ROS_DOMAIN_ID=0 >> ~/.bashrc  
source ~/.bashrc
```

Connecter ensuite tous les appareils au même réseau, tous les topics seront alors accessibles à tous les appareils du réseau. Cela permet de faire un contrôle centralisé des différents robots. Par exemple, pour un essaim, un ordinateur central pourra directement publier sur les topics *cmd_vel* attribués à chaque module.

6 Motion Capture

6.1 Installation du package *vrpn_mocap*

La section suivante décrit comment récupérer, sur un topic ROS2, la position des *Rigid Bodies* détectés par le système de Motion Capture Motive.

Commencer par créer un nouvel espace de travail ROS2. Ouvrir un terminal et entrer les commandes suivantes pour créer le répertoire *mocap_ws* ainsi que son sous-répertoire *src* :

```
mkdir -p ~/mocap_ws/src  
cd ~/mocap_ws
```

Cloner ensuite le dépôt GitHub du package *vrpn_mocap* dans le répertoire *src* de l'espace de travail :

```
cd ~/mocap_ws/src  
git clone https://github.com/alvinsunyixiao/vrpn_mocap.git
```

Une fois le dépôt cloné, installer les dépendances nécessaires à la compilation en exécutant la commande suivante :

```
rosdep install --from-paths src -y --ignore-src
```

Procéder ensuite à la compilation du package :

```
colcon build
```

Enfin, sourcer l'espace de travail :

```
source install/setup.bash
```

Pour éviter d'avoir à sourcer l'environnement à chaque nouvelle session, ajouter cette ligne au fichier *.bashrc* :

```
echo "source ~/mocap_ws/install/setup.bash" >> ~/.bashrc
```

Il est possible de personnaliser le comportement du package en modifiant le fichier *client.launch.yaml*. Les principaux paramètres configurables sont les suivants :

- *server* : adresse IP ou nom de domaine du serveur VRPN (par défaut : *localhost*).
- *port* : port utilisé par le serveur VRPN (par défaut : *3883*).
- *frame_id* : nom de la frame de référence fixe (par défaut : *world*).
- *update_freq* : fréquence de publication des données de capture de mouvement (par défaut : *100.0 Hz*).

Pour tester le package, utiliser la commande suivante (modifier l'adresse IP et le port pour correspondre au système de Motion Capture) :

```
ros2 launch vrpn_mocap client.launch.yaml server:=192.168.0.103 port  
↪ :=3883
```


Il est possible de modifier le fichier *client.launch.yaml*, afin de ne pas avoir à spécifier l'IP du serveur et le port dans la commande. Pour cela, modifier le fichier *client.launch.yaml* se trouvant dans */mocap/src/vrpn_mocap/launch* pour qu'il corresponde à :

```
launch:

- arg:
  name: "server"
  default: "192.168.0.103"
- arg:
  name: "port"
  default: "3883"

- node:
  pkg: "vrpn_mocap"
  namespace: "vrpn_mocap"
  exec: "client_node"
  name: "vrpn_mocap_client_node"
  param:
  -
    from: "${(find-pkg-share vrpn_mocap)/config/client.yaml}"
  -
    name: "server"
    value: "${(var server)}"
  -
    name: "port"
    value: "${(var port)}"
```

La nouvelle commande à utiliser pour lancer le noeud sera alors :

```
ros2 launch vrpn_mocap client.launch.yaml
```

6.2 Quality of Service (QoS) sur ROS2

Les paramètres de *Quality of Service (QoS)* sur ROS2 permettent de contrôler la manière dont les données sont échangées entre les nœuds. Par exemple, le paramètre *reliability* spécifie la garantie de livraison des messages entre les publishers et les subscribers. Deux options sont disponibles pour ce paramètre :

- **Best Effort** : tente de livrer les messages, mais certains peuvent être perdus si le réseau est instable. Ce mode est analogue au protocole **UDP**, qui privilégie la rapidité d'envoi des paquets sans garantie de réception.
- **Reliable** : garantit la livraison des messages, quitte à effectuer plusieurs tentatives en cas d'échec. Ce comportement est similaire à **TCP**, qui assure la fiabilité de la transmission en retransmettant les données en cas de perte.

Cependant, ces deux modes ne sont pas toujours compatibles entre eux. Par exemple, un *subscriber*

configuré en *Reliable* ne pourra pas recevoir les messages d'un *publisher* en mode *Best Effort*, car il s'attend à une garantie stricte de recevoir des données. En ROS2, les *publisher* et *subscriber* sont initialement réglés en *Reliable*, et ne peuvent donc pas afficher les messages publiés par *vrpn_mocap* qui sont en *Best Effort*. Pour régler ce problème, il faut, si possible, modifier les configurations QoS (voir Annexe A).

Ci-dessous un tableau récapitulatif des compatibilités du paramètre *Reliable* :

Publisher	Subscriber	Compatible
Best Effort	Best Effort	Oui
Best Effort	Reliable	Non
Reliable	Best Effort	Oui
Reliable	Reliable	Oui

TABLE 6.1 – Compatibilité du paramètre *Reliable*

Si modifier les paramètres n'est pas possible (comme par exemple sur rqt), il est nécessaire de d'ajouter la ligne ci-dessous dans le fichier *client.yaml*, situé dans *mocap_ws/src/vrpn_mocap/config*, afin de forcer le paramètre *Reliability* de *vrpn_mocap* en *Reliable* :

```
sensor_data_qos: false
```

Le fichier *client.yaml* final doit ressembler à ceci :

```
/vrpn_mocap/vrpn_mocap_client_node:
  ros__parameters:
    server: localhost
    port: 3883
    frame_id: world
    update_freq: 100.
    refresh_freq: 1.
    sensor_data_qos: false
    multi_sensor: false
    use_vrpn_timestamps: false
```

Puis, *build* à nouveau pour être sûr de prendre en compte les modifications.

```
colcon build
source install/setup.bash
```

Une alternative, dans le cas où l'utilisation du mode *Best Effort* est impérative pour la publication, consiste à créer un nœud intermédiaire contenant un *subscriber* configuré en *Best Effort*. Ce dernier souscrita aux topics fournis par *vrpn_mocap* et republiera les messages à l'aide d'un *publisher* configuré en *Reliable*, sur de nouveaux topics. Cela permet ainsi d'assurer la compatibilité avec les outils nécessitant une QoS fiable, tout en conservant les topics réglés en *Best Efforts*.

Des informations supplémentaires sur les QoS sont disponibles ici :
<https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Quality-of-Service-Settings.html>

7 TF2

7.1 Définition

tf2 est une bibliothèque de ROS2 permettant de gérer plusieurs repères de coordonnées en parallèle. Cela est particulièrement utile lorsque différents composants d'un système utilisent chacun leur propre repère. Par exemple, un système de Motion Capture fournit la position d'un robot dans un repère global fixe, tandis que chaque robot dispose de son propre repère local, centré sur lui-même. Se déplacer selon l'axe x du repère de Motion Capture ne correspond donc pas nécessairement à un mouvement vers l'avant pour le robot.

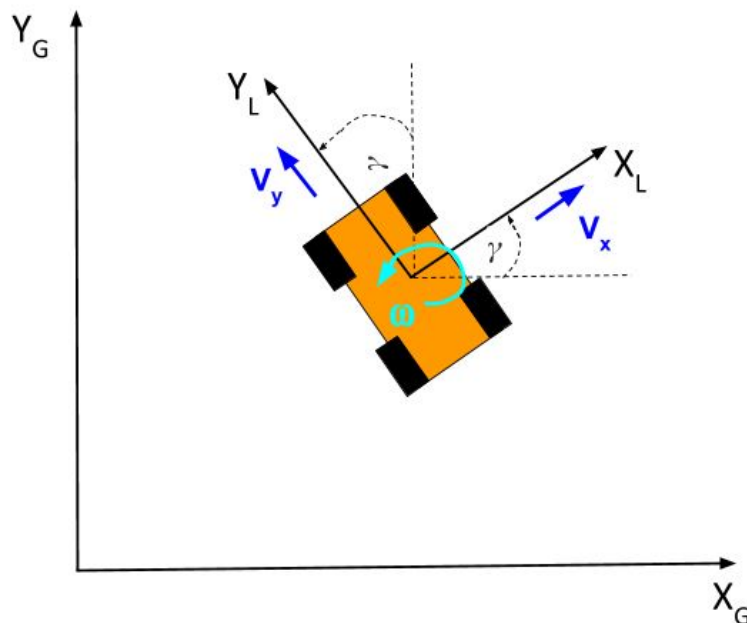


FIGURE 7.1 – Repère globale et repère Body

Prenons l'exemple Figure 7.1, le robot est piloté par des commandes de vitesses linéaires exprimées dans son repère local X_L, Y_L . Si une commande de vitesse $v_x > 0$ est envoyée, le robot avancera dans la direction de son axe X_L , quelle que soit son orientation dans le repère global. Toutefois, si tous les calculs (planification, contrôle en formation, navigation, etc.) sont effectués dans le repère global X_G, Y_G , il est alors nécessaire de transformer ces vitesses depuis le repère global vers le repère local du robot avant de les lui envoyer. Inversement, les positions ou vitesses mesurées dans le repère local doivent être transformées dans le repère global pour être comparées ou utilisées dans les algorithmes centraux.

tf2 permet donc de convertir les positions, orientations ou vitesses d'un repère à un autre, ce qui permet de piloter ou d'observer le robot dans le repère qui est le plus pertinent pour la tâche à accomplir.

7.2 Mise en place de tf2

7.2.1 Initialisation de tf2 : Buffer, Listener et Broadcaster

Dans un premier temps, il faut initialiser trois composants principaux :

- **Buffer** : stocke les transformations entre repères.
- **TransformListener** : écoute les transformations publiées sur le réseau ROS et alimente le buffer.
- **TransformBroadcaster** : permet de publier de nouvelles transformations (par exemple, la position du robot dans le repère global).

```
from tf2_ros import Buffer, TransformListener, TransformBroadcaster

class MyNode(Node):
    def __init__(self):
        super().__init__('my_tf2_node')
        self.tf_buffer = Buffer()
        self.tf_listener = TransformListener(self.tf_buffer, self)
        self.tf_broadcaster = TransformBroadcaster(self)
```

Listing 7.1 – Initialisation de tf2 dans un nœud ROS2

7.2.2 Publication de la transformation du robot

À chaque réception d'une nouvelle pose (par exemple depuis un système de Motion Capture), il faut publier la transformation entre le repère global (ex : *mocap*) et le repère du robot (ex : *robot_name/base_link*).

```
from geometry_msgs.msg import TransformStamped

def pose_callback(self, msg):
    transform = TransformStamped()
    transform.header.stamp = self.get_clock().now().to_msg()
    transform.header.frame_id = "mocap" # repere global
    transform.child_frame_id = "robot_name/base_link" # repere du robot

    transform.transform.translation.x = msg.pose.position.x
    transform.transform.translation.y = msg.pose.position.y
    transform.transform.translation.z = msg.pose.position.z

    transform.transform.rotation.x = msg.pose.orientation.x
    transform.transform.rotation.y = msg.pose.orientation.y
    transform.transform.rotation.z = msg.pose.orientation.z
    transform.transform.rotation.w = msg.pose.orientation.w

    self.tf_broadcaster.sendTransform(transform)
```

Listing 7.2 – Publication de la transformation à partir de la pose

7.2.3 Transformation d'une commande de vitesse

Pour piloter le robot dans son repère local à partir de commandes exprimées dans le repère global, il faut changer la base dans lequel le vecteur est exprimé. Pour cela, un message *Vector3Stamped* et la méthode *lookup_transform* sont utilisés pour obtenir la transformation entre les deux repères.

```
from geometry_msgs.msg import Vector3Stamped
import tf2_geometry_msgs
from tf2_ros import TransformException

def transform_velocity(self, vx, vy, vz=0.0):
    global_vel = Vector3Stamped()
    global_vel.header.frame_id = "mocap"
    global_vel.header.stamp = self.get_clock().now().to_msg()
    global_vel.vector.x = vx
    global_vel.vector.y = vy
    global_vel.vector.z = vz

    try:
        transform = self.tf_buffer.lookup_transform(
            "robot_name/base_link", # frame cible
            "mocap", # frame source
            rclpy.time.Time(),
            timeout=rclpy.duration.Duration(seconds=0.1)
        )
        robot_vel = tf2_geometry_msgs.do_transform_vector3(global_vel,
        ↪ transform)
        return robot_vel.vector.x, robot_vel.vector.y, robot_vel.vector.
        ↪ z
    except TransformException as ex:
        self.get_logger().error(f"Erreur de transformation TF2 : {ex}")
        return vx, vy, vz # fallback : pas de transformation
```

Listing 7.3 – Transformation d'une vitesse du repère global au repère du robot

7.2.4 Exemple d'utilisation

Soit les trois robots *Aramis*, *Athos* et *Porthos* positionnés dans l'espace comme sur la photo ci-dessous :

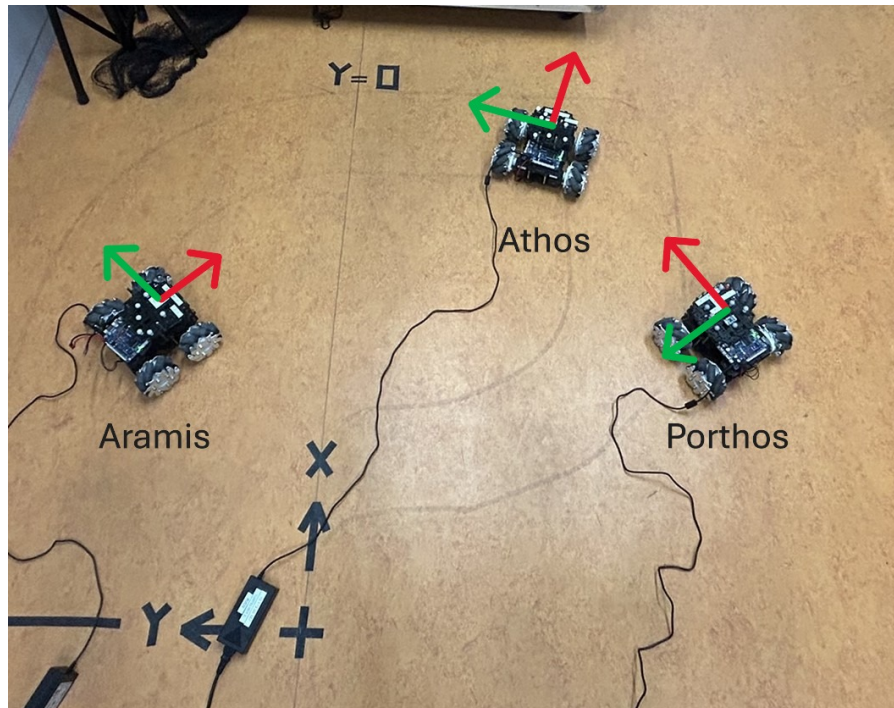


FIGURE 7.2 – Démonstration tf2

En utilisant tf2, et un outil de visualisation tel que RViz, il est alors possible de visualiser les 3 repères correspondants aux trois robots comme montré ci-dessous.

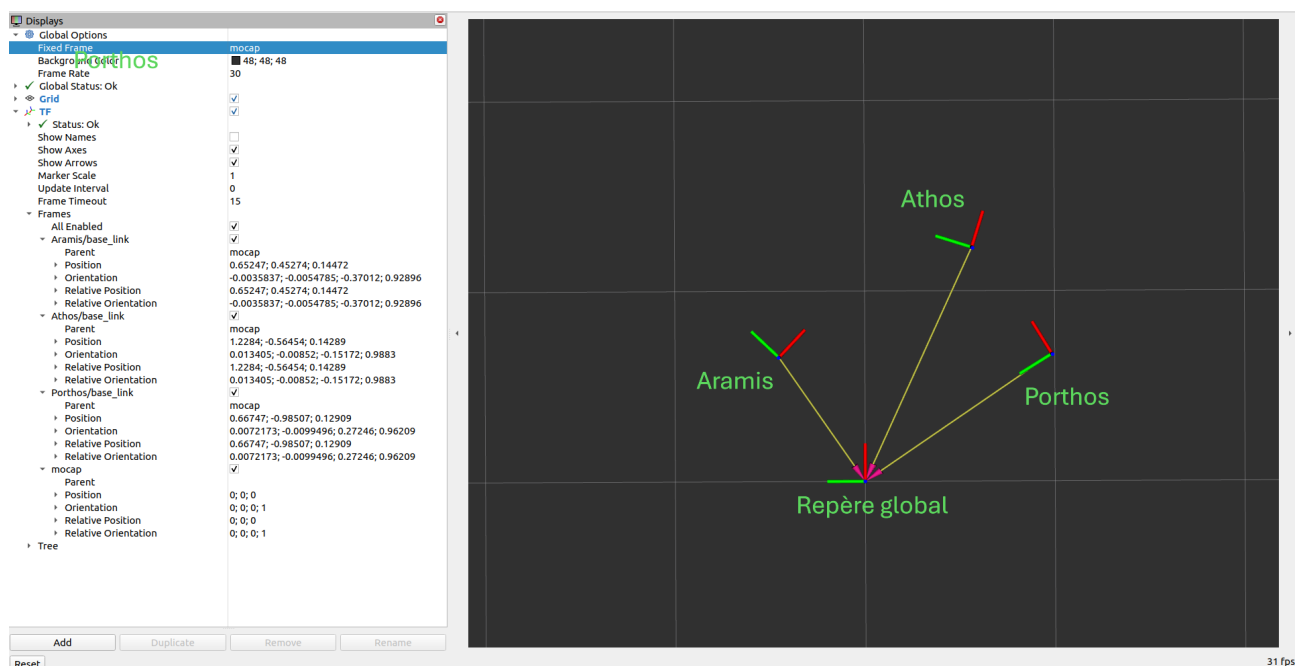


FIGURE 7.3 – Visualisation RViz

Note : Pour lancer Rviz, utiliser la commande suivante :

```
ros2 run rviz2 rviz2 -d $(ros2 pkg prefix --share turtle_tf2_py)/rviz/  
↪ turtle_rviz.rviz
```

8 Package *Mecanum*

8.1 Téléchargement du package

L'ensemble des programmes pour utiliser les robots Mecanum se trouvent sur Github. Pour les utiliser, cloner le repository suivant :

```
git clone https://github.com/watson67/mecanum.git
```

Puis, se placer dans le dossier cloné, et utiliser les commandes suivantes :

```
cd mecanum/src  
rm -rf build/install/log/  
colcon build --symlink-install --cmake-target clean  
colcon build --symlink-install  
source install/setup.bash
```

Pour ne pas avoir à sourcer le fichier *setup.bash* à chaque ouverture de terminal, utiliser la commande suivante :

```
echo "source ~/mecanum/install/setup.bash" >> ~/.bashrc
```

8.2 Utilisation du package

Un tutoriel équivalent au format *markdown* est disponible sur le Github suivant : https://github.com/watson67/mecanum/tree/main?tab=readme-ov-file#teleop_mecanum.

Table des figures

1.1	Raspberry Pi Imager	2
3.1	Lancement de l'agent	7
3.2	Lancement de l'agent avec OpenCR	7
3.3	Pinout OpenCR	8
4.1	URL de gestionnaire de cartes supplémentaires	10
7.1	Repère globale et repère Body	17
7.2	Démonstration tf2	20
7.3	Visualisation Rviz	20
A.1	Paramétrage des QoS sur Simulink	25

A Modification des QoS

A.1 Python

Dans Python, les QoS sont configurés avec des objets *QoSProfile*. Voici un exemple de création d'un publisher avec une fiabilité *Best Effort* ([Source :Exemple PX4 Offboard Python](#)) :

```
from rclpy.qos import QoSProfile, QoSReliabilityPolicy
import rclpy
from std_msgs.msg import String

rclpy.init()

node = rclpy.create_node('qos_publisher')

qos_profile = QoSProfile(
    depth=10,
    reliability=QoSReliabilityPolicy.BEST Effort
)

publisher = node.create_publisher(String, 'topic_name', qos_profile)
```

Listing A.1 – Publisher avec QoS personnalisé

Pour le subscriber :

```
from rclpy.qos import QoSProfile, QoSReliabilityPolicy

qos_profile = QoSProfile(
    depth=10,
    reliability=QoSReliabilityPolicy.BEST Effort
)

subscriber = node.create_subscription(
    String,
    'topic_name',
    callback,
    qos_profile
)
```

Listing A.2 – Subscriber avec QoS personnalisé

A.2 Matlab/Simulink

A.2.1 Matlab

Source : Documentation Matlab

```
node = ros2node('matlab_node');
pub = ros2publisher(node, '/topic_name', 'std_msgs/String', ...
    'Depth', 10, 'Reliability', 'besteffort');
```

Listing A.3 – Publisher ROS 2 Matlab

```
sub = ros2subscriber(node, '/topic_name', 'std_msgs/String', ...
    'Depth', 10, 'Reliability', 'besteffort', ...
    'Callback', @(msg)disp(msg.data));
```

Listing A.4 – Subscriber ROS 2 Matlab

A.2.2 Simulink

Dans Simulink, les blocs ROS Publish et ROS Subscribe peuvent aussi être configurés avec des paramètres QoS :

- Double-cliquer sur le bloc *Subscribe* ou *Publish*.
- Dans l'onglet **QoS Settings**, il est possible de modifier les QoS

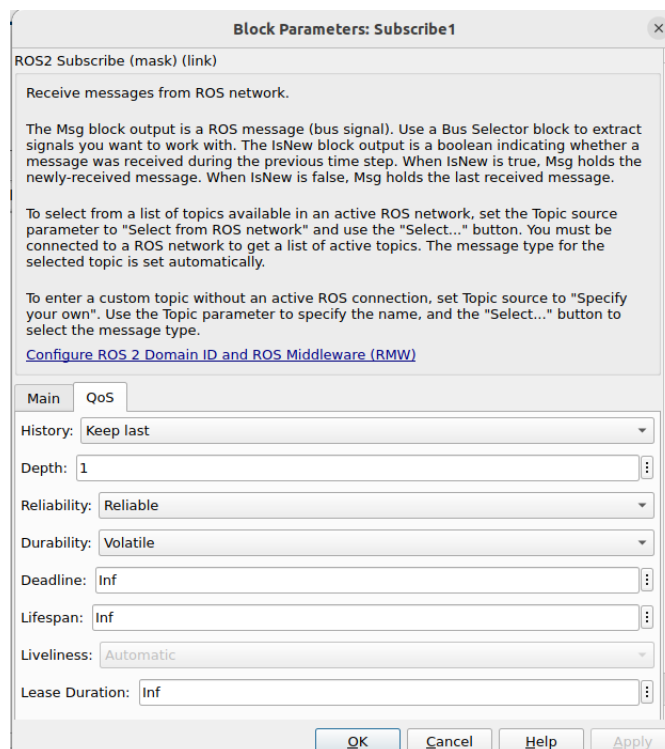


FIGURE A.1 – Paramétrage des QoS sur Simulink

B Changement de *hostname* et renommage de l'utilisateur principal

Introduction

Cette partie s'adresse aux utilisateurs d'Ubuntu Server, ayant créé une image d'Ubuntu dans le but de la copier sur une nouvelle Raspberry Pi. Nous verrons comment modifier de façon durable le *hostname*, car il est nécessaire d'en avoir des différents sur chaque robot pour utiliser les programmes du workspace *mecanum*. Nous verrons également comment renommer proprement l'utilisateur principal pour correspondre au nouveau *hostname*.

B.1 Création d'un utilisateur temporaire

Avant de modifier le *hostname* principal ou de renommer un utilisateur, il est recommandé de créer un utilisateur temporaire disposant des droits *sudo* pour ne pas perdre l'accès administrateur en cas de problème.

```
sudo adduser tempadmin  
sudo usermod -aG sudo tempadmin
```

Déconnectez-vous, puis reconnectez-vous en tant que *tempadmin* pour poursuivre les modifications. Pour cela, vous pouvez soit redémarrer la Raspberry Pi, soit utiliser les commandes suivantes :

```
exit  
ssh tempadmin@ip_de_votre_machine
```

B.2 Arrêter tous les processus de l'utilisateur à renommer

Pour éviter des conflits lors du renommage, arrêtez tous les processus de l'utilisateur à modifier (ici *dartagnan*) :

```
sudo pkill -u dartagnan
```

B.3 Renommer l'utilisateur principal

1. Renommer le login

```
sudo usermod -l mercedes dartagnan
```

2. Renommer le dossier personnel

```
sudo usermod -d /home/mercedes -m mercedes
```

3. Renommer le groupe principal (optionnel)

```
sudo groupmod -n mercedes dartagnan
```

4. Assurer les bonnes permissions sur le dossier personnel

```
sudo chown -R mercedes:mercedes /home/mercedes
```

B.4 Modifier la configuration de *cloud-init*

Commencez par éditer le fichier de configuration principal de *cloud-init* :

```
sudo nano /etc/cloud/cloud.cfg
```

Localisez la ligne contenant *preserve_hostname : false* et remplacez-la par :

```
preserve\_hostname: true
```

Cela indique à *cloud-init* de ne pas modifier le hostname au démarrage. Sauvegardez et quittez l'éditeur.

B.5 Modifier les fichiers */etc/hostname* et */etc/hosts*

Modifiez le fichier */etc/hostname* pour indiquer le nouveau nom souhaité (par exemple *mercedes-desktop*) :

```
sudo nano /etc/hostname
```

Changez le contenu en remplaçant l'ancien hostname par le nouveau, puis sauvegardez.

Modifiez aussi le fichier */etc/hosts* pour mettre à jour la correspondance avec le nouveau hostname :

```
sudo nano /etc/hosts
```

Recherchez la ligne contenant l'ancien hostname, par exemple :

```
127.0.1.1    mercedes
```

et remplacez-la par :

```
127.0.1.1    mercedes-desktop
```

Sauvegardez et fermez l'éditeur.

B.6 Vérification optionnelle de la configuration Netplan

Bien que rarement problématique, il est utile de vérifier si Netplan ne force pas un hostname au démarrage.

Listez les fichiers de configuration Netplan :

```
ls /etc/netplan/
```

Ouvrez le fichier YAML principal (le nom peut varier, ici *50-cloud-init.yaml*) :

```
sudo nano /etc/netplan/50-cloud-init.yaml
```

Si vous trouvez une ligne *set-hostname* ou *hostname*, commentez-la ou supprimez-la.

B.7 Redémarrage

Pour appliquer toutes les modifications, redémarrez la machine :

```
sudo reboot
```

Après redémarrage, votre invite de commande devrait refléter le nouveau hostname et utilisateur, par exemple :

```
mercedes@mercedes-desktop:~$
```